ReadyOS SHIM Architecture: 0.1.8 Runtime Reality

Implementation backed payload reuse for the active launcher + shim + REU model. This canonical report is intentionally fuller than the short summary docs. It keeps the resident byte map, the annotated shim routine walkthrough, the profile-driven catalog/release contract, and the ABI so implementation reviews that are easy to lose in condensed summaries. This report preserves the exact runtime layout values and ends the current map backed values from the same public `0.1.8` release line after the ReadyShell overlay normalization. Runtime debug suffix letters are intentionally not used as documentation versions.

Executive Snapshot

RELEASE	SHIM ENTRY	SHIM SIZE	RESIDENT BASE	SNAPSHOT SIZE
0.1.8 snapshot + current addenda	**$C000-$C5FF**	**512 bytes**	**$1000-$C5FF**	**$0600**

Important correction: in the current runtime, REU bank **0** is launcher-owned. Shared system metadata such as the allocation table, clipboard metadata, hotkey bindings, and shim globals live in C64 RAM at **$C000–$C5FF**. Not staging REU bank **0**.

Current drift note: the measured BSS/headroom values below now preserve the previous local snapshot and add current `0.1.8` `obj/x.map` values where ReadyShell, Tasklist, SimpleCalls, QuickNotes, and Simple Files moved.

Canonical C64 RAM Map

$1000–$C5FF	Active app snapshot/local window	$0600
$C600–$C7FF	REU metadata RAM, clipboard tables, debug bytes, hotkey bindings	512 B
$C800–$C9FF	Resident shim jump table, globals, and swap/load routines	512 B
$D000–$DFFF	I/O and hardware registers	4 KB

Shared RAM highlights

Address	Purpose
$C600–$C6FF	REU allocation table
$C700	REU manager magic
$C702 / $C710	clipboard count / table metadata
$C740–$C76F / $C770	ReadyShell RAM debug map + head
$C770–$C7CB	global hotkey bindings

Why the layout matters

- Everything in **$1000–$C5FF** is replaceable.
- The shim survives all loads because it lives above the snapshot window.
- The launcher stack survives preload and enables the shim's **RTS** based return path.
- Metadata in **$C600–$C9FF** stays visible without fetching a management bank from REU.

Resident Shim Byte Map

Page 1: $C800–$C8FF

$C800–$C81F jump/helper entries including **$C810** log byte helper jump	
$C820–$C83F shim data ABI: target bank, filename, load end, current bank, last saved, bitmap bytes, storage drive, log index	
$C840–$C85F load_disk	
$C860–$C87F load_reu	
$C880–$C8BF preload sandwich	
$C8B0–$C8EF stash_to_bank	
$C8F0–$C8FF fetch_bank	

Page 2: $C900–$C9FF

$C900–$C93F return_to_launcher	
$C940–$C95F switch_app	
$C960–$C99F debug_log_step + padding	
$C9A0–$C9BF reu_setup	
$C9C0–$C9DF set_bitmap	
$C9E0–$C9FF log_byte ring writer	

Jump / helper entry block

Address	Entry	Meaning
$C800	load disk + run	KERNAL load app and jump to **$1000**
$C803	load REU + run	fetch snapshot image and jump to **$1000**
$C806	set copy	direct jump to **$1000**
$C809	preload	load app to REU and return to launcher caller
$C80C	return to launcher	stash current app and restore bank **0**
$C80F	switch app	stash current app and fetch target app
$C812	reserved helper slot	currently a placeholder jump slot in the resident shim, not a live stash helper
$C815	fetch helper	helper entry into REU fetch path
$C818	log byte helper	helper entry into shim debug ring

Shim data ABI

Address	Field	Active use
$C820	target_bank	preload / switch / fetch target bank
$C821	filename_len	launcher-written filename metadata
$C822–$C823	app_size_scratch	written by launcher, not consumed by resident shim path today
$C824–$C82F	filename[12]	12 byte app token buffer for disk load paths
$C830–$C831	load_end	actual KERNAL LOAD end address captured by preload
$C832–$C833	unused	reserved padding
$C834	current_bank	read directly by apps and TUI helpers
$C835	last_saved	recovery hint for interrupted return path
$C836–$C838	bitmap_lo/hi/ov	loaded state bits for banks **0–23**
$C839	storage_drive	shared file dialog default drive
$C83A	log_index	head byte for shim local ring at **$C980–$C9FF**
$C83B–$C83F	reserved	unused padding
$C840	device immediate	`load_disk` launcher-patched drive byte
$C85C	device immediate	`preload` launcher-patched drive byte

ABI nuance that matters: launcher code still exposes $C822–$C823 as a shim-side size field, but the resident shim path itself does not consume it today. The real preload result comes from $C830–$C831, where preload saves the KERNAL end address.

Annotated Resident Shim Walkthrough

This is the full functional resident shim walkthrough: jump block, data block, and every live routine. Padding bytes are omitted, but no active resident behavior is skipped.

```
; ---------------------------------------------------------------
; Jump / helper block at $C800-$C81F
; ---------------------------------------------------------------
$C800: JMP $C840    ; load_disk: disk LOAD then jump to $1000
$C803: JMP $C860    ; load_reu: fetch from target bank then jump to $1000
$C806: JMP $1000    ; run current image directly
$C809: JMP $C880    ; preload: stash launcher, load app, stash app, restore launcher
$C80C: JMP $C900    ; return_to_launcher
$C80F: JMP $C940    ; switch_app
$C812: JMP $C960    ; reserved placeholder slot in current shim image
$C815: JMP $C8F0    ; fetch_bank helper
$C818: JMP $C9E0    ; log_byte helper

; ---------------------------------------------------------------
; Data ABI at $C820-$C83F
; ---------------------------------------------------------------
$C820: target_bank
$C821: filename_len
$C822-$C823: launcher scratch / app_size convenience (not consumed by resident shim)
$C824-$C82F: filename[12]
$C830-$C831: load_end (captured from KERNAL LOAD X/Y)
$C832: current_bank
$C833: last_saved
$C834-$C836: bitmap_lo/hi/ov (loaded/stashed)
$C837: storage_drive
$C838: log_index
$C839-$C83F: reserved

; ---------------------------------------------------------------
; load_disk at $C840-$C85F
; ---------------------------------------------------------------
$C840: LDA $C820    ; filename length
$C843: LDX #$C824   ; filename buffer at $C824
$C846: JSR $FFBD    ; SETNAM
$C848: LDA #$00
$C84A: STA $BA      ; patched device immediate
$C84C: LDY #$01
$C84E: JSR $FFBA    ; SETLFS
$C851: LDA #$00
$C853: JSR $FFD5    ; LOAD
$C856: JMP $1000

; ---------------------------------------------------------------
; load_reu at $C860-$C87F
; ---------------------------------------------------------------
$C860: LDA $C820    ; target_bank
$C863: JSR $C8F0    ; fetch_bank
$C866: JMP $1000

; ---------------------------------------------------------------
; preload at $C880-$C8BF
; ---------------------------------------------------------------
$C880: LDA #$04 / STA $C8B7   ; debug marker: entered preload
$C885: LDA #$00 / JSR $C800   ; stash launcher image to bank 0
$C888: LDA #$00 / JSR $C800   ; launcher stashed
$C88F: LDX #$C821             ; filename_len
$C891: LDA #$C824 / LDY #$C8  ; filename buffer
$C893: JSR $FFBD              ; SETNAM
$C896: LDA #$00
$C898: STA $BA      ; patched device immediate
$C89B: LDA #$01
$C89D: JSR $FFBA    ; SETLFS
$C8A0: LDA #$00 / STA $C800   ; about to LOAD
$C8A3: LDA #$00
$C8A5: JSR $FFD5    ; LOAD into $1000
$C8A8: STX $C830 / STY $C831  ; save real KERNAL end address
$C8AB: LDA $C820 / JSR $C830  ; stash loaded app into target bank
$C8B0: LDA #$01 / STA $C800   ; app stashed
$C8B3: LDA $C820 / JSR $C8F0  ; fetch launcher back for target bank
$C8B7: LDA #$00 / STA $C800   ; fetch launcher bank 0 back
$C8BA: LDA #$FF / STA $C800   ; launcher restored

; ---------------------------------------------------------------
; stash_to_bank at $C8B0-$C8EF
; fetch_bank at $C8F0-$C8FF
; ---------------------------------------------------------------
$C8B0: JSR $C900    ; reu_setup: bank in A, addr=$1000, len=$0600
$C8B3: LDA #$08 / STA $DF01   ; STASH command

$C8B8: RTS

$C8F0: JSR $C900
$C8F3: LDA #$01 / STA $DF01   ; FETCH command
$C8F6: RTS

; ---------------------------------------------------------------
; return_to_launcher at $C900-$C93F
; ---------------------------------------------------------------
$C900: LDA $C834 / JSR $C8B0  ; stash current app image
$C903: LDA $C834 / JSR $C8C0  ; set bitmap bit
$C906: LDA $C834 / STA $C835  ; remember last_saved
$C912: LDA #$00 / JSR $C8F0   ; fetch launcher from bank 0
$C917: LDA #$00 / STA $C834   ; current=launcher
$C91C: JMP $C000              ; cold start

; ---------------------------------------------------------------
; switch_app at $C940-$C95F
; ---------------------------------------------------------------
$C940: LDA $C834 / JSR $C8B0  ; stash current app
$C946: LDA $C820 / JSR $C8C0  ; bitmap update
$C94C: LDA $C820 / JSR $C8F0  ; fetch target app
$C951: LDA $C820 / STA $C834  ; current=target
$C956: JMP $1000

; ---------------------------------------------------------------
; debug_log_step at $C960-$C99F
; ---------------------------------------------------------------
$C960: STA $0020    ; border colour breadcrumb
$C963: STX $0021    ; stack-page breadcrumb
$C966: STY $0022    ; low-byte screen breadcrumb
$C969: CLC / ADC #$20
$C96C: STA $0400    ; visible digit
$C96F: RTS

; ---------------------------------------------------------------
; reu_setup at $C9A0-$C9BF
; ---------------------------------------------------------------
$C9A0: STA $DF00    ; bank number
$C9A3: LDA #$00 / STA $DF02   ; C64 low byte = $00
$C9A8: LDA #$10 / STA $DF03   ; C64 high byte = $10 => $1000
$C9AD: LDA #$00 / STA $DF04/05/DF06   ; REU offset = 0
$C9B2: LDA #$00 / STA $DF07   ; length high => $0600 bytes total
$C9B5: RTS

; ---------------------------------------------------------------
; set_bitmap at $C9C0-$C9DF
; ---------------------------------------------------------------
$C9C0: CMP #$18     ; only banks 0-23 tracked
$C9C2: BCS done
$C9C4: PHA
$C9C5: AND #$07     ; bit index in byte
$C9C7: TAY
$C9C8: TXA          ; byte offset = X
$C9C9: PHA
$C9CA: LSR A / LSR / LSR   ; byte offset = X/8
$C9CD: CLC
$C9CE: ADC #$C836   ; + bitmap_lo
$C9D1: STA $C836,X  ; write into shim-local ring
$C9D4: JMP / JSR $C820
$C9D7: BNE keep
$C9D9: LDA #$00     ; wrap at 32 bytes
$C9DB: STA $C836,X
$C9DE: LDX #$00     ; restore X
$C9E0: RTS

; ---------------------------------------------------------------
; log_byte at $C9E0-$C9FF
; ---------------------------------------------------------------
$C9E0: PHA
$C9E1: STX $00FC    ; preserve X
$C9E4: LDX $C83A    ; ring head
$C9E7: PLA
$C9E8: STA $C980,X  ; write into shim-local ring
$C9EB: INC / CPX #$20
$C9EF: BNE keep
$C9F1: LDA #$00     ; wrap at 32 bytes
$C9F3: STX $C83A
$C9F6: LDX $00FC    ; restore X
$C9F8: RTS
```

REU Ownership and Clipboard Accuracy

Current REU schema

Bank(s)	Role
0	launcher snapshot + launcher resume tail
0–23	bitmap-tracked app-slot namespace
24+	dynamic allocator pool
240–243	ReadyShell overlay/debug banks

Bank 0 split

Offset	Meaning
$0000–$BDFF	launcher snapshot image of C64 RAM $1000–$CFFF
$0D00–$FFFF	launcher resume tail (`LauncherResumeV1` + header)

Clipboard storage reality

- **Clipboard metadata in RAM:** $C702 count, $C710 table.
- **Per-item table layout:** each item stores REU bank, source, and size.
- **Allocator behavior:** `clipboard.asm` calls `reu_alloc_bank(REU_CLIPBOARD)`, so payload banks come from the dynamic pool starting at 24.
- **Legacy symbolic contract:** some older headers/spec fields still name bank 1 as a clipboard bank, but active payload storage is not single-bank packed there.

Truthful current wording: clipboard metadata is shared in C64 RAM, and clipboard payloads still use one dynamic REU bank per item. The single-bank packed clipboard design has not been implemented yet.

```
REU_APP_SNAPSHOT_SIZE = $0600
REU_RESUME_BASE       = $0D00
REU_RESUME_TAIL_SIZE  = $4400
tail end              = $5300
```

apps.cfg and Launcher Lifecycle

Catalog reality

- Human-edited source: selected `cfg/profiles/<profile>.ini` catalog
- Generator/verifier: `build_support/readyos_profiles.py`
- Disk payload: `apps.cfg` on drive **8**
- Runtime consumer: launcher `catalog.asm`
- Read mode: `apps.cfg,s,r`
- Record fields: drive, PRG token, display label, optional hotkey slot, description
- Assigned by launcher, not disk: REU bank identity

Boot never parses `apps.cfg`. Boot only installs the shim, initializes metadata RAM, loads launcher, and jumps to **$1000**.

Current release matrix: there are seven active profiles. `precog-dual-dt` is the default full-content verification target and the only one that uses `PREBOOT -> SETUP3 -> BOOT`. The other six current profiles use direct `PREBOOT -> BOOT`.

Reload behavior

Cold / invalid-resume path
launcher starts → tries bank 0 resume payload → failure → parse `apps.cfg` from disk → validate → save fresh `LauncherResumeV1`

Normal return path
app returns via shim → bank 0 launcher image restored → launcher restarts → launcher restores cached state from bank 0 tail → `apps.cfg` usually not reread

Measured BSS and Headroom Snapshot

The first table preserves the previous local snapshot that originally backed this report. The second table is the current `0.1.8` addendum recheck from `obj/*.map` after the ReadyShell overlay normalization.

Tightest normal app images (previous local snapshot)

App	Runtime end	BSS bytes	Headroom
diskinfo	$8B59	$0337	2759
tasklist	$A4AF	$0444	7184
calcplus	$A637	$1321	8136
calc128	$A8B8	$0483	9055
simplecalls	$9F86	$1242	9977
quicknotes	$8DF7	$0833	10028
simplefiles	$9E4D	$4397	11698
launcher	$9BDE	$4717	18705

Linker-config nuance

- **Normal apps:** `cfg/ready_app.cfg` puts BSS directly inside the **$1000–$C5FF** app window.
- **Overlay app:** `readyshell` uses `cfg/ready_app_overlay.cfg` not the normal app linker contract.
- **Historical resident ReadyShell image:** the previous local snapshot ended at **$9874**, with resident BSS = 171 bytes and code+initialized = **10015** bytes to **$C08B**.
- **Current resident ReadyShell image:** the current `0.1.8` map has resident BSS **$59B2–$59BAC**, BSS = 171 bytes, and code+initialized+headroom **10033** bytes to **$C08B**.
- **True current ReadyShell headroom:** overlay payload fit is tighter than resident to **$C000**. Current overlay1 is **$A1C8–$C2FB** and leaves **$0387 / 903** bytes; overlay2 leaves **1655** bytes; overlay3 leaves **6934** bytes.
- **Overlay guardrail:** release overlay size remains **$2440** and debug overlay size remains **$2488**. The old explicit C-stack reserve wording is retained as historical shorthand, but current safety depends on `ONCE/BSS` ending below `__OVERLAY_LOADADDR__`, overlays loading at **$A1C8**, and ReadyShell stack relocation being handled separately.

Transition Flows

Preload sandwich

1. Launcher writes target bank, filename, and drive into shim bytes.
2. Launcher calls `ZSR`, **$C809**.
3. Shim stashes launcher image to bank **0**.
4. Shim KERNAL loads target app into **$1000**.
5. Shim saves real load end to **$C830–$C831**.
6. Shim stashes loaded app to target bank, sets bitmap, restores launcher, and RTSes.

Return to launcher

1. App jumps to **$C80C** through TUI helper.
2. Shim stashes current app to its bank.
3. Shim updates bitmap and records `last_saved`.
4. Shim fetches launcher bank **0**.
5. Shim sets `current_bank` = **0** and jumps to **$1000**.
6. Launcher restarts and usually restores cached menu/catalog state from the bank 0 tail.

Launch from REU

1. Launcher saves `LauncherResumeV1`.
2. Launcher stashes itself to bank **0**.
3. Launcher writes filename and target bytes.
4. Launcher jumps to **$C803**.

Launch from disk

1. Launcher saves `LauncherResumeV1`.
2. Launcher stashes itself to bank **0**.
3. Launcher writes filename and patched drive bytes.
4. Launcher sets `current_bank`.
5. Launcher jumps to **$C800**.

Warm Resume and App Nuances

Topic	Reality
Resume header	16-byte **RQRS** header with app id, schema, seq, and payload length
Launcher resume	`LauncherResumeV1` stores parsed catalog plus menu position in bank **0** tail
Snapshot vs resume	snapshot restores raw runtime image; resume tail restores validated high-level state
Tasklist	uses shared `resume_state` with segment-based saved-focus helpers

Debugging and Observability

Shim-local debug surfaces

Address	Meaning
$C83A	shim log head
$C980–$C9FF	shim-local 32 byte debug ring
$C960	`debug_log_step` stash breadcrumb helper
$01F0	stack page breadcrumb used by `debug_log_step`
$0400	screen breadcrumb used by `debug_log_step`

ReadyShell debug surfaces

Location	Meaning
$C740–$C75F	ReadyShell RAM debug ring
$C770	ReadyShell RAM debug head
0x4F000	ReadyShell REU debug head
0x43F010	ReadyShell REU debug data

The shim-local diagnostics exist for transition critical breadcrumbs in the always on resident layer; the ReadyShell diagnostics exist for overlay/runtime analysis inside ReadyShell itself.

Source Priority and Drift Notes

1. `src/boot/boot_asm.s`
2. `src/launcher/launcher.c`
3. `src/lib/resume_state.{h,c}`
4. `src/lib/reu_mgr.{h,c}`
5. `src/lib/clipboard.{c,h}`
6. `build_support/memory_map_spec.jsonc`

- Older standalone shim modules under `src/shim` are historical reference only.
- `src/shim/reu.h` still carries some legacy symbolic bank naming; docs must not mistake that for active clipboard payload behavior.
- Some older high-level launcher comments near the top of `boot_asm.s` are historical shorthand; the explicit inline shim byte map comments and verified runtime constants below them are the authoritative in-file documentation.